



LAB REPORT

Course Code: CSE2202

Course Name: Algorithm Analysis and Design Sessional

Experiment No: 2

Experiment Name: Calculating Efficiency of different sorting Algorithms.

Submitted By: Name: Shad Ibna Hosain Roll: 2103131 Section: C Session: 2021-22	Submitted To: A. F. M. Minhazur Rahman Assistant Professor Rajshahi University of Engineering and Technology
--	--

Task 1

Problem Statement:

The Quick hull and Gramhamscan algorithm is used to find the outer most point in the 2D plane among other points and through that hull points get the minimum area in where any of random two points can be connected, means the direct connection line will be in the area generated by adding the hull points directly. So, ultimately the two algorithm try to find the hull points efficiently. There has been used the divide and conquere process in quick hull and stack data structure has been used basically in the grahamscan algorithm.

Code:

Quick Hull:

<pre>#include <bits/stdc++.h> #include <chrono> using namespace std; using namespace std::chrono; #define ll long long vector<pair<ll, ll>> randomarr() { vector<pair<ll, ll>> v; ll n = 30; for (int i = 0; i < n; i++) { ll p1 = rand() % n; ll p2 = rand() % n; v.push_back(make_pair(p1, p2)); } return v; } ll getS(pair<ll, ll>& p1, pair<ll, ll>& p2, pair<ll, ll>& p) { ll crossP = (p2.first - p1.first) * (p.second - p1.second) - (p2.second - p1.second) * (p.first - p1.first); if (crossP > 0) return 1; else if (crossP < 0) return -1; else return 0; }</pre>	<pre>void orie(pair<ll, ll>& p1, pair<ll, ll>& p2, vector<pair<ll, ll>>& result, vector<pair<ll, ll>>&down, vector<pair<ll, ll>>&up, vector<pair<ll, ll>>&rem){ for(auto x:result){ ll ck = getS(p1, p2, x); if(ck==-1)up.push_back(x); else if(ck==1)down.push_back(x); else rem.push_back(x); } } int main() { srand(time(0)); vector<pair<ll, ll>> v = randomarr(); vector<pair<ll, ll>> result; pair<ll, ll> p1 = *min_element(v.begin(), v.end()); pair<ll, ll> p2 = *max_element(v.begin(), v.end()); convexHull(p1, p2, v, result); convexHull(p2, p1, v, result); vector<pair<ll, ll>> down; vector<pair<ll, ll>> up; vector<pair<ll, ll>> rem; rem.push_back(p1);</pre>
--	--

<pre> ll dis(pair<ll, ll>& p1, pair<ll, ll>& p2, pair<ll, ll>& p) { return abs((p2.first - p1.first) * (p1.second - p.second) - (p1.first - p.first) * (p2.second - p1.second)); } void convexHull(pair<ll, ll>& p1, pair<ll, ll>& p2, vector<pair<ll, ll>>& v, vector<pair<ll, ll>>& ans) { pair<ll, ll> md = {0, 0}; ll mx = 0; bool got = false; for (auto x : v) { ll ck = getS(p1, p2, x); if (ck == 1) { ll d = dis(p1, p2, x); if (d > mx) { md = x; mx = d; got = true; } } } if (!got) return; ans.push_back(md); convexHull(p1, md, v, ans); convexHull(md, p2, v, ans); } </pre>	<pre> orie(p1,p2,result,down,up,rem); rem.push_back(p2); sort(down.begin(), down.end(), [](const pair<ll, ll>& a, const pair<ll, ll>& b) { if (a.first != b.first) return a.first < b.first; return a.second > b.second; }); sort(up.begin(), up.end(), [](const pair<ll, ll>& a, const pair<ll, ll>& b) { if (a.first != b.first) return a.first > b.first; return a.second < b.second; }); result.clear(); for(int i=0;i<rem.size()-1;i++){ result.push_back(rem[i]); } for(int i=0;i<down.size();i++){ result.push_back(down[i]); } result.push_back(rem[rem.size()-1]); for(int i=0;i<up.size();i++){ result.push_back(up[i]); } for (auto& point : v) { cout << "(" << point.first << ", " << point.second << ")"<<" "; } cout<<endl; for (auto& point : result) { cout << "(" << point.first << ", " << point.second << ")"<<" "; } return 0; } </pre>
--	--

Graham scan 1 :

<pre> #include <bits/stdc++.h> #include <chrono> using namespace std; using namespace std::chrono; #define ll long long typedef pair<ll, ll> two; vector<two> randomarr() { vector<two> v; ll n = 100; for (int i = 0; i < n; i++) { ll p1 = rand() % n; ll p2 = rand() % n; v.push_back(make_pair(p1, p2)); } return v; } two findLowestPoint(vector<two>& v) { two lowest = v[0]; for (auto& q : v) { if (q.second < lowest.second (q.second == lowest.second && q.first < lowest.first)) { lowest = q; } } return lowest; } ll getS(two p1, two p2, two p) { ll crossP = (p2.first - p1.first) * (p.second - p1.second) - (p2.second - p1.second) * (p.first - p1.first); if (crossP > 0) return 1; else if (crossP < 0) return -1; else return 0; } ll dis(two& p1, two& p2, two& p) { return abs((p2.first - p1.first) * (p1.second - p.second) - (p1.first - p.first) * (p2.second - p1.second)); } int crossProduct(two ref, two a, two b) { </pre>	<pre> two p1(stack<two> s) { return s.top(); } two p2(stack<two> s) { s.pop(); return s.top(); } void graham(stack<two>& s, vector<two>& v) { for (int i = 2; i < v.size(); i++) { ll ck = getS(p2(s), p1(s), v[i]); if (ck == 1) { s.push(v[i]); } else if (ck==0){ s.pop(); s.push(v[i]); } else if (ck == -1) { s.pop(); while (getS(p2(s), p1(s), v[i]) != 1) { s.pop(); } s.push(v[i]); } } } int main() { srand(time(0)); vector<two> v = randomarr(); stack<two> s; sortPointsByPolarAngle(v); s.push(v[0]); s.push(v[1]); graham(s, v); for (auto& point : v) { cout << "(" << point.first << ", " << point.second << ")" << ", "; } } </pre>
---	---

<pre> return (a.first - ref.first) * (b.second - ref.second) - (a.second - ref.second) * (b.first - ref.first); } bool polarCompare(two ref, two a, two b) { int cross = crossProduct(ref, a, b); if (cross == 0) { int distA = (a.first - ref.first) * (a.first - ref.first) + (a.second - ref.second) * (a.second - ref.second); int distB = (b.first - ref.first) * (b.first - ref.first) + (b.second - ref.second) * (b.second - ref.second); return distA < distB; } return cross > 0; } void sortPointsByPolarAngle(vector<two>& v) { two ref = findLowestPoint(v); sort(v.begin(), v.end(), [ref](two a, two b) { return polarCompare(ref, a, b); }); } </pre>	<pre> cout << endl; vector<two> hull; while (!s.empty()) { hull.push_back(s.top()); s.pop(); } //reverse(hull.begin(), hull.end()); for (auto& point : hull) { cout << "(" << point.first << ", " << point.second << ")" << ","; } return 0; } </pre>
---	--

Graham Scan 2 : (A more efficient than previous)

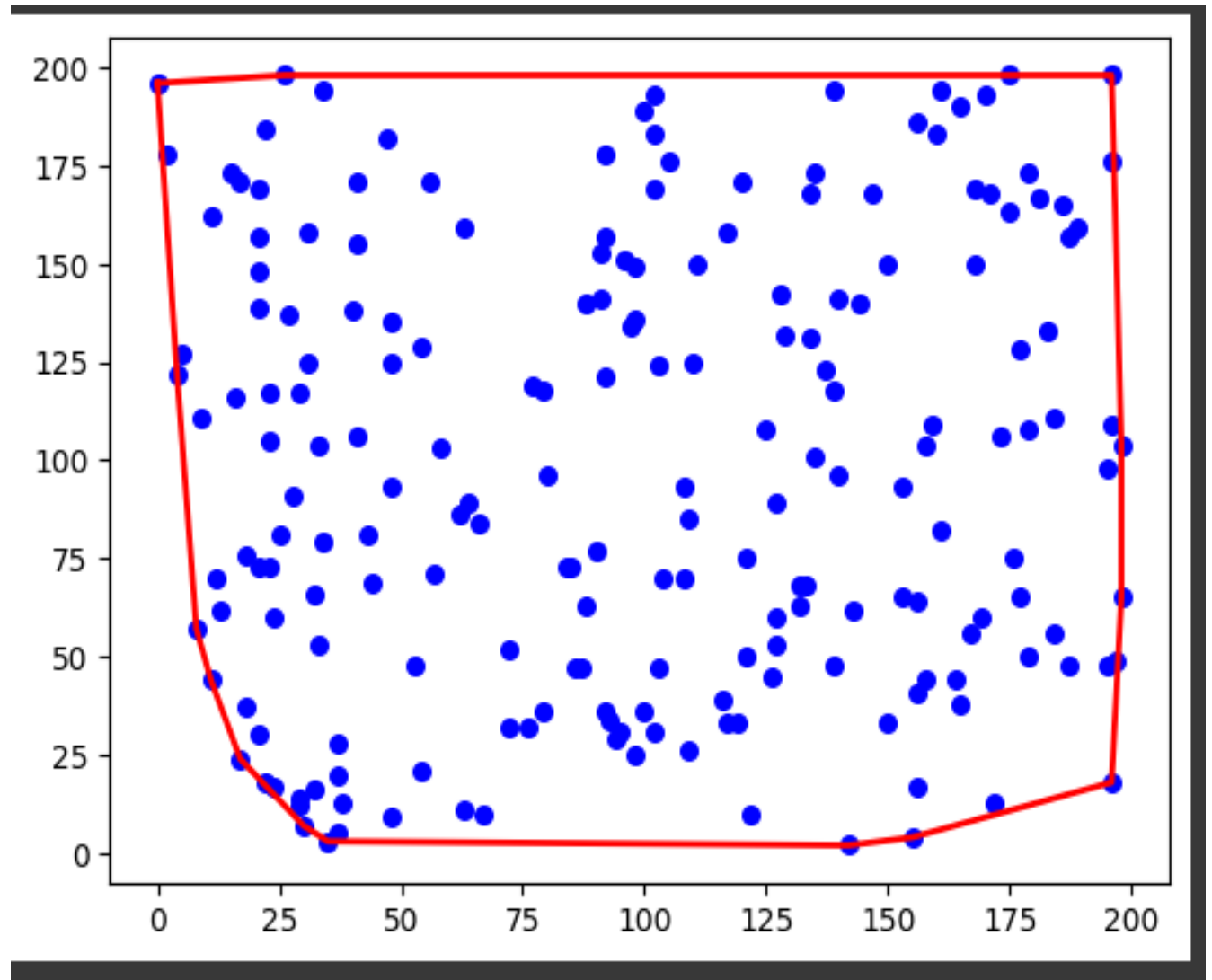
<pre> #include <bits/stdc++.h> #include <chrono> using namespace std; using namespace std::chrono; #define ll long long typedef pair<ll, ll> two; vector<two> randomarr() { vector<two> v; ll n = 1000000; </pre>	<pre> two secondTop(stack<two>& s) { two top = s.top(); s.pop(); two second = s.top(); s.push(top); return second; } </pre>
--	---

<pre> for (int i = 0; i < n; i++) { ll p1 = rand() % n; ll p2 = rand() % n; v.push_back(make_pair(p1, p2)); } return v; } two findLowestPoint(vector<two>& v) { return *min_element(v.begin(), v.end(), [](two a, two b) { return a.second < b.second (a.second == b.second && a.first < b.first); }); } ll getS(two p1, two p2, two p) { return (p2.first - p1.first) * (p.second - p1.second) - (p2.second - p1.second) * (p.first - p1.first); } ll dis(two p1, two p2) { return (p1.first - p2.first) * (p1.first - p2.first) + (p1.second - p2.second) * (p1.second - p2.second); } </pre>	<pre> void graham(stack<two>& s, vector<two>& v) { s.push(v[0]); s.push(v[1]); for (size_t i = 2; i < v.size(); i++) { while (s.size() > 1 && getS(secondTop(s), s.top(), v[i]) <= 0) { s.pop(); } s.push(v[i]); } } int main() { srand(time(0)); vector<two> v = randomarr(); auto start = high_resolution_clock::now(); sortPointsByPolarAngle(v); stack<two> s; graham(s, v); auto stop = high_resolution_clock::now(); auto duration = duration_cast<microseconds>(st op - start); cout << "Execution Time: " << duration.count() << " microseconds" << endl; </pre>
--	--

<pre>void sortPointsByPolarAngle(vector<two >& v) { two ref = findLowestPoint(v); sort(v.begin(), v.end(), [ref](two a, two b) { ll cross = getS(ref, a, b); if (cross == 0) { return dis(ref, a) > dis(ref, b); } return cross > 0; }); v.erase(unique(v.begin(), v.end()), v.end()); }</pre>	<pre>vector<two> hull; while (!s.empty()) { hull.push_back(s.top()); s.pop(); } for (auto& point : hull) { cout << "(" << point.first << ", " << point.second << ") "; } return 0; }</pre>
--	--

Output Visualization:

Quick Hull:



Graham scan:

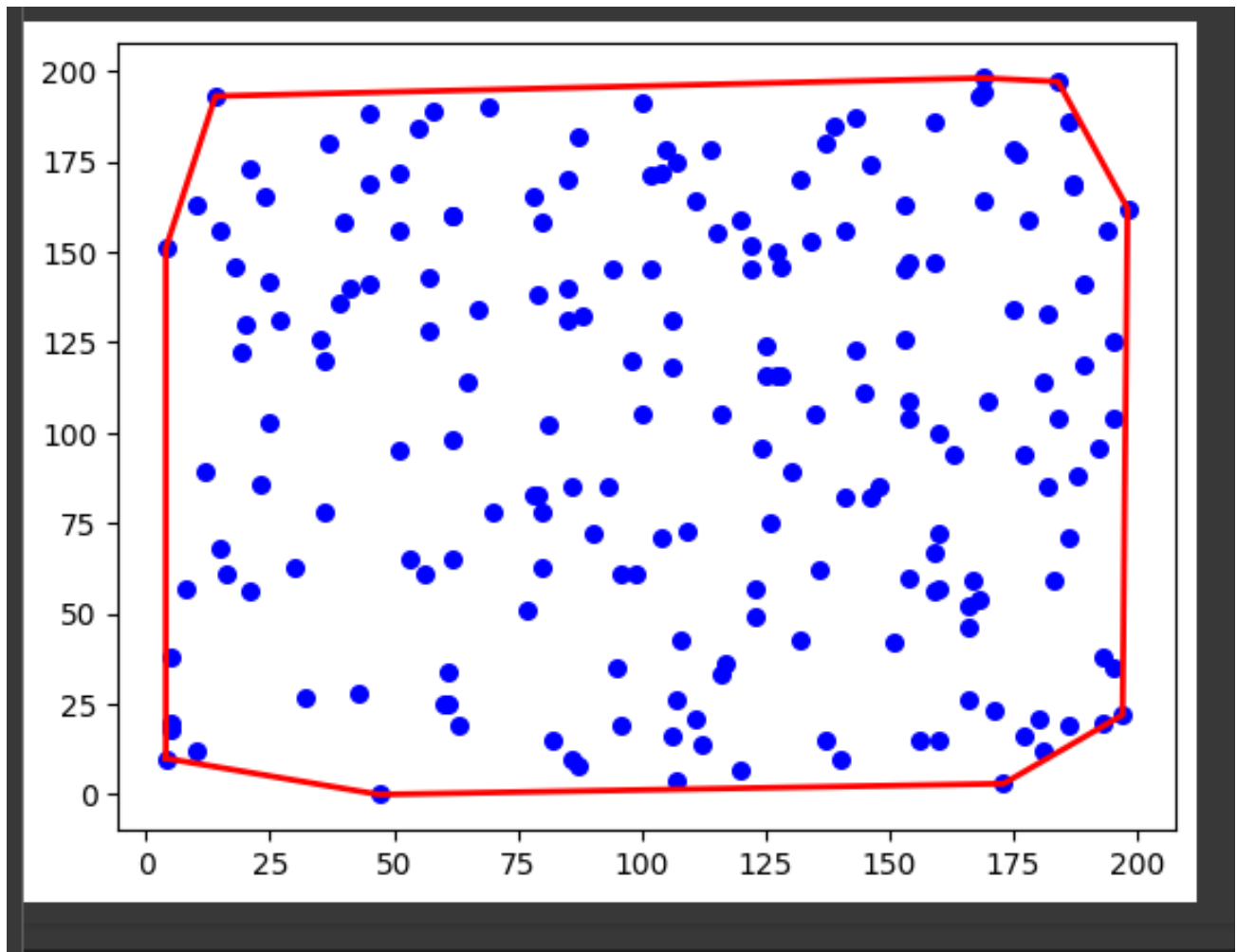


Table of comparison between Quick hull and Graham scan algorithm execution time :
All time are measures in microseconds

N	Quick Hull	Graham Scan 1	Graham Scan 2
10^2	0	800	0
10^3	0	3997	0
10^4	9003	30994	7992
10^5	57082	231998	53916
10^6	414000	2219992	616008
10^7	3763045	-----	7118521

Results and Discussion:

First of all here is two graham scan code Graham scan 1 and Graham Scan 2. In the table it is clearly visible that graham scan 2 is much more efficient compare to graham scan 1 the reason is that in the graham scan 1 the stack was not being perfectly used as well as the duplicate points has been removed before the stack call causing the algorithm graham scan 2 work much fast. There has been also a great change that when the points are linear ,that has been found earlier, keeping the most farthest point and rest were being removed. These all optimization causing the Graham Scan 2 being much efficient. I checked my Graham Scan 1 and found out these issue and now it works far better.

Now comes the next observable point that after 100k data size or from there the graham scan being slower than quick hull algorithm. Where Graham Scan takes almost constantly $O(n \log n)$ time complexity where Quick Hull takes on average $O(n \log n)$ to worst case $O(n^2)$. In the best case when the points are all collinear then it takes $O(n)$ times but its completely rare case. I figure out that when there are a lots of points between two points and if the duplicate points are not that much then if two farthest points are taken which is a key operation for Quick hull algo then it seems to be the other points are kind of being closes to the line joining p1 and p2 making a collinear line (which is not actually that but if it is seen from naked eye the plotted graph).so the execution is being less than Graham Scan. Where Graham Scan always sort the points in $O(n \log n)$ times. So being a lots of points in a congested area causing the Quick Hull faster than Graham Scan.